EECE 5639: Computer Vision
Fall 2026, Homework 3

Jingqi Mei

October 23, 2026

Normal form of a line: $x\cos\theta + y\sin\theta = \rho$, where ρ is the signed distance and $0^\circ \le \theta < 180^\circ$.

Question 1

(a)

- AB: $A = (2, 0)$, $B = (2, 2)$, so the line is $x = 2$. $\quad \Rightarrow (\rho_1, \theta_1) = (2,\ 0^\circ)$
- BC: $B = (2, 2)$, $C = (0, 2)$, so the line is $y = 2$. $\quad \Rightarrow (\rho_2, \theta_2) = (2,\ 90^\circ)$
- CA: $C = (0, 2)$, $A = (2, 0)$, so the line is $x + y = 2$. Dividing by $\sqrt{2}$ gives $x\cos 45^\circ + y\sin 45^\circ = \sqrt{2}$. $\quad \Rightarrow (\rho_3, \theta_3) = (\sqrt{2},\ 45^\circ)$

(b)

Line	(ρ_i, θ_i)	$(\rho_i^2,\ \theta_i + 90^\circ)$	(ρ_i', θ_i') in range	Equation
1	$(2, 0^\circ)$	$(4, 90^\circ)$	$(4, 90^\circ)$	$y = 4$
2	$(2, 90^\circ)$	$(4, 180^\circ)$	$(-4, 0^\circ)$	$x = -4$
3	$(\sqrt{2}, 45^\circ)$	$(2, 135^\circ)$	$(2, 135^\circ)$	$-x + y = 2\sqrt{2}$

Line 2 has $\theta = 180^\circ$, which is outside the range. Since (ρ, θ) and $(-\rho, \theta - 180^\circ)$ describe the same line, it becomes $(-4, 0^\circ)$.

Intersecting the three lines gives the vertices:

$$A' = L_1 \cap L_2 = (-4,\ 4),$$
$$B' = L_1 \cap L_3 = (4 - 2\sqrt{2},\ 4) \approx (1.17,\ 4),$$
$$C' = L_2 \cap L_3 = (-4,\ 2\sqrt{2} - 4) \approx (-4,\ -1.17).$$

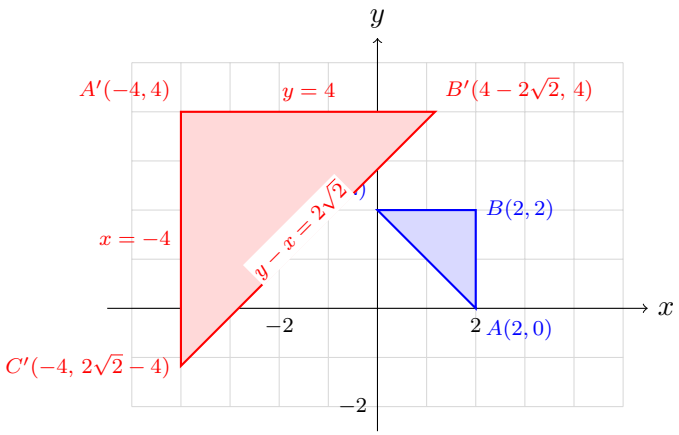


Figure 1: Original triangle (blue) and modified triangle (red).

(c)

- Original: a right triangle at B with legs $|AB| = |BC| = 2$, so the area is $\frac{1}{2} \cdot 2 \cdot 2 = \mathbf{2}$.
- Modified: a right triangle at A' with legs $|A'B'| = |A'C'| = 8 - 2\sqrt{2}$, so the area is

$$\tfrac{1}{2}\left(8 - 2\sqrt{2}\right)^2 = \mathbf{36 - 16\sqrt{2}} \approx \mathbf{13.37}.$$

Question 2

(a)

The four lines are:

- AB: $y = 3$
- BC: $x = 3$
- CD: $\frac{1}{2}x + \frac{\sqrt{3}}{2}y = \sqrt{3}$, i.e. $x + \sqrt{3}\,y = 2\sqrt{3}$
- DA: $-\frac{1}{\sqrt{2}}x + \frac{1}{\sqrt{2}}y = \frac{1}{\sqrt{2}}$, i.e. $y = x + 1$

The vertices are the intersections of adjacent sides:

$$A = DA \cap AB = (2,\ 3), \qquad B = AB \cap BC = (3,\ 3),$$
$$C = BC \cap CD = (3,\ 2 - \sqrt{3}) \approx (3,\ 0.27), \qquad D = CD \cap DA = \left(\tfrac{3-\sqrt{3}}{2},\ \tfrac{5-\sqrt{3}}{2}\right) \approx (0.63,\ 1.63).$$

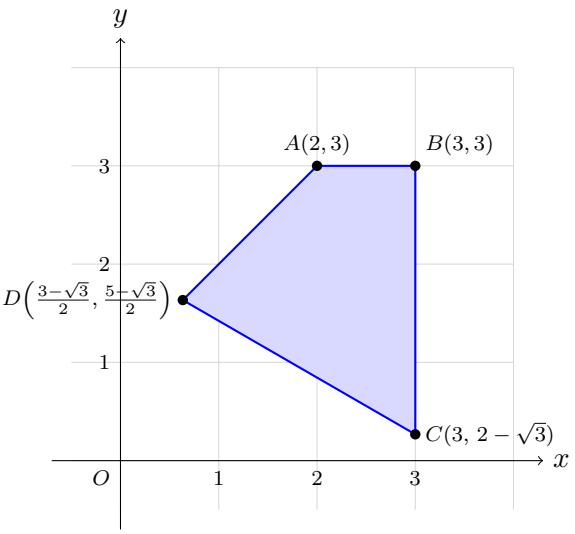


Figure 2: The convex polygon $ABCD$.

(b)

Let $\mathbf{n}_\theta = (\cos\theta, \sin\theta)$. A point $\mathbf{p}$ is on the line when $\mathbf{n}_\theta \cdot \mathbf{p} = \rho$. Rotating by 90° gives $R\mathbf{p}$, and $\mathbf{n}_{\theta+90^\circ} \cdot R\mathbf{p} = \mathbf{n}_\theta \cdot \mathbf{p} = \rho$. So each line maps to $(\rho,\ \theta + 90^\circ)$. If the new angle is $\geq 180^\circ$, we subtract 180° and flip the sign of ρ:

Side	(ρ, θ)	$(\rho, \theta + 90^\circ)$	(ρ', θ')	Rotated line
$A'B'$	$(3, 90^\circ)$	$(3, 180^\circ)$	$(-3,\ 0^\circ)$	$x = -3$
$B'C'$	$(3, 0^\circ)$	$(3, 90^\circ)$	$(3,\ 90^\circ)$	$y = 3$
$C'D'$	$(\sqrt{3}, 60^\circ)$	$(\sqrt{3}, 150^\circ)$	$(\sqrt{3},\ 150^\circ)$	$-\frac{\sqrt{3}}{2}x + \frac{1}{2}y = \sqrt{3}$
$D'A'$	$(\frac{1}{\sqrt{2}}, 135^\circ)$	$(\frac{1}{\sqrt{2}}, 225^\circ)$	$(-\frac{1}{\sqrt{2}},\ 45^\circ)$	$x + y = -1$

(c)

Rotation about the origin does not change the distance from the origin to a line, so $|\rho|$ stays the same. The normal direction rotates with the line, so θ increases by the rotation angle ($\theta' = \theta + 90^\circ$ here). In Hough space this is a shift along the θ axis. When θ' goes past 180°, it wraps around and the sign of ρ flips.

Question 3

Points: $(0,0), (1,2), (2,1), (3,3)$, $n = 4$.
$\sum x_i = 6$, $\sum y_i = 6$, $\sum x_i^2 = 14$, $\sum x_i y_i = 13$, $\quad \bar{x} = \bar{y} = 1.5$.

(a) Ordinary least squares

Setting $\partial J_{\text{OLS}}/\partial m = 0$ and $\partial J_{\text{OLS}}/\partial b = 0$ gives the normal equations:

$$\begin{cases} m\sum x_i^2 + b\sum x_i = \sum x_i y_i \\ m\sum x_i + n\, b = \sum y_i \end{cases} \quad \Rightarrow \quad \begin{cases} 14m + 6b = 13 \\ 6m + 4b = 6 \end{cases}$$

From the second equation, $b = 1.5 - 1.5m$. Substituting: $14m + 9 - 9m = 13$, so $m = 0.8$ and $b = 0.3$.

$$\boxed{y = 0.8x + 0.3}, \qquad J_{\text{OLS}} = \tfrac{1}{4}(0.3^2 + 0.9^2 + 0.9^2 + 0.3^2) = 0.45.$$

(b) Total least squares

Step 1: find c. Setting $\partial J_{\text{TLS}}/\partial c = \frac{2}{4}\sum(ax_i + by_i + c) = 0$ gives $c = -(a\bar{x} + b\bar{y})$. The line therefore passes through the centroid $(1.5, 1.5)$, and

$$J_{\text{TLS}} = \tfrac{1}{4}\sum \left(a(x_i - \bar{x}) + b(y_i - \bar{y})\right)^2 = \mathbf{n}^T \Sigma\, \mathbf{n}, \qquad \mathbf{n} = (a, b)^T.$$

Step 2: covariance matrix. The centred points are $(-1.5, -1.5)$, $(-0.5, 0.5)$, $(0.5, -0.5)$, $(1.5, 1.5)$, so

$$\Sigma = \frac{1}{4}\begin{bmatrix} \sum \tilde{x}^2 & \sum \tilde{x}\tilde{y} \\ \sum \tilde{x}\tilde{y} & \sum \tilde{y}^2 \end{bmatrix} = \frac{1}{4}\begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 1.25 & 1 \\ 1 & 1.25 \end{bmatrix}.$$

Step 3: eigen-analysis. Minimizing $\mathbf{n}^T\Sigma\mathbf{n}$ subject to $\|\mathbf{n}\| = 1$ with a Lagrange multiplier gives $\Sigma\mathbf{n} = \lambda\mathbf{n}$ and $J_{\text{TLS}} = \lambda$. So $\mathbf{n}$ must be the eigenvector with the *smallest* eigenvalue. From $(1.25 - \lambda)^2 - 1 = 0$:

$$\lambda_1 = 0.25, \;\; \mathbf{n}_1 = \tfrac{1}{\sqrt{2}}(1, -1)^T; \qquad \lambda_2 = 2.25, \;\; \mathbf{n}_2 = \tfrac{1}{\sqrt{2}}(1, 1)^T.$$

Therefore $a = \frac{1}{\sqrt{2}}$, $b = -\frac{1}{\sqrt{2}}$, and $c = -\frac{1}{\sqrt{2}}(1.5 - 1.5) = 0$:

$$\boxed{\tfrac{1}{\sqrt{2}}x - \tfrac{1}{\sqrt{2}}y = 0 \;\; \Leftrightarrow \;\; y = x}, \qquad J_{\text{TLS}} = \lambda_{\min} = 0.25.$$

(c)

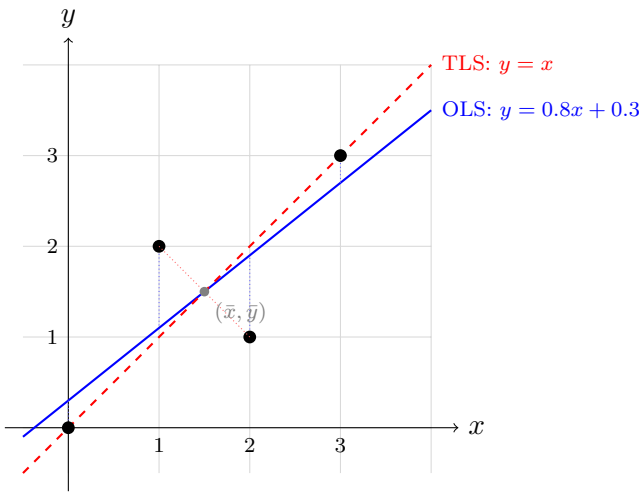


Figure 3: Data, OLS line (blue) with its vertical residuals, and TLS line (red) with its perpendicular residuals.

The two methods minimize different errors. OLS minimizes *vertical* distances: it treats x as exact and puts all the error in y. TLS minimizes *perpendicular* distances, treating x and y the same way. The data are symmetric about $y = x$, so TLS returns exactly $y = x$. The OLS slope $m = S_{xy}/S_{xx} = 4/5 = 0.8$ is pulled toward 0 by the scatter in y. Both lines pass through the centroid $(1.5, 1.5)$.

(d)

Total least squares. When the noise in x and y is independent with equal variance, the error is isotropic. The best estimate (the maximum-likelihood fit) then minimizes perpendicular distance, which is what TLS does. TLS also does not depend on how the coordinate axes are oriented, and it handles vertical edges, where OLS fails because the slope is infinite.

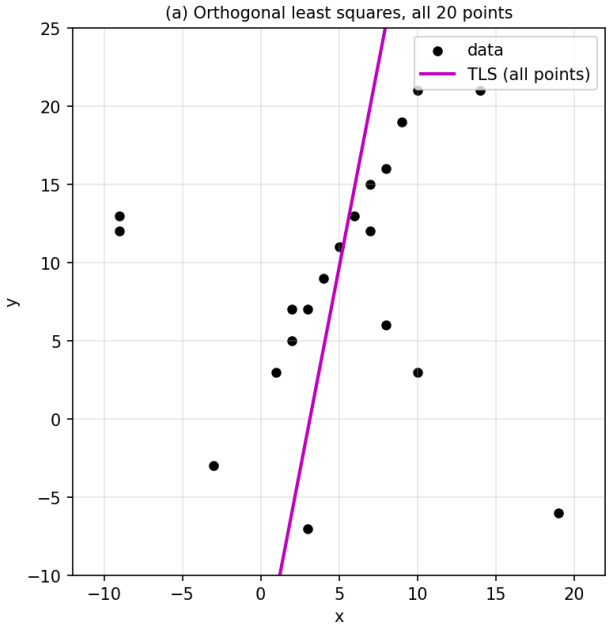
Question 4

Code: `hw3_q4.py` (Appendix). TLS uses the smallest eigenvector of the covariance matrix, as in Question 3(b).

(a)

For all 20 points: $(\bar{x}, \bar{y}) = (4.85,\ 8.85)$, $\Sigma = \begin{bmatrix} 43.43 & 3.88 \\ 3.88 & 62.83 \end{bmatrix}$, with eigenvalues 42.68 and 63.57.

$$-0.9820\,x + 0.1890\,y + 3.0900 = 0 \quad (y = 5.196x - 16.349).$$



(b)

The random seed is 0. The inlier threshold is 1.5, with $p = 0.99$, $s = 2$ and $N_{\max} = 100$. Whenever the best inlier count improves, the iteration limit becomes $\min\big(\lceil \ln(1-p)/\ln(1-w^2) \rceil,\ 100\big)$:

Iteration	Inliers (new best)	w	New limit N
1	4	0.20	$\min(113, 100) = 100$
2	6	0.30	49
4	13	0.65	9

Iterations 6 and 8 also found 13 inliers, but with a larger sum of squared distances (5.12 and 3.31 vs. 3.19), so iteration 4 is kept as the best.

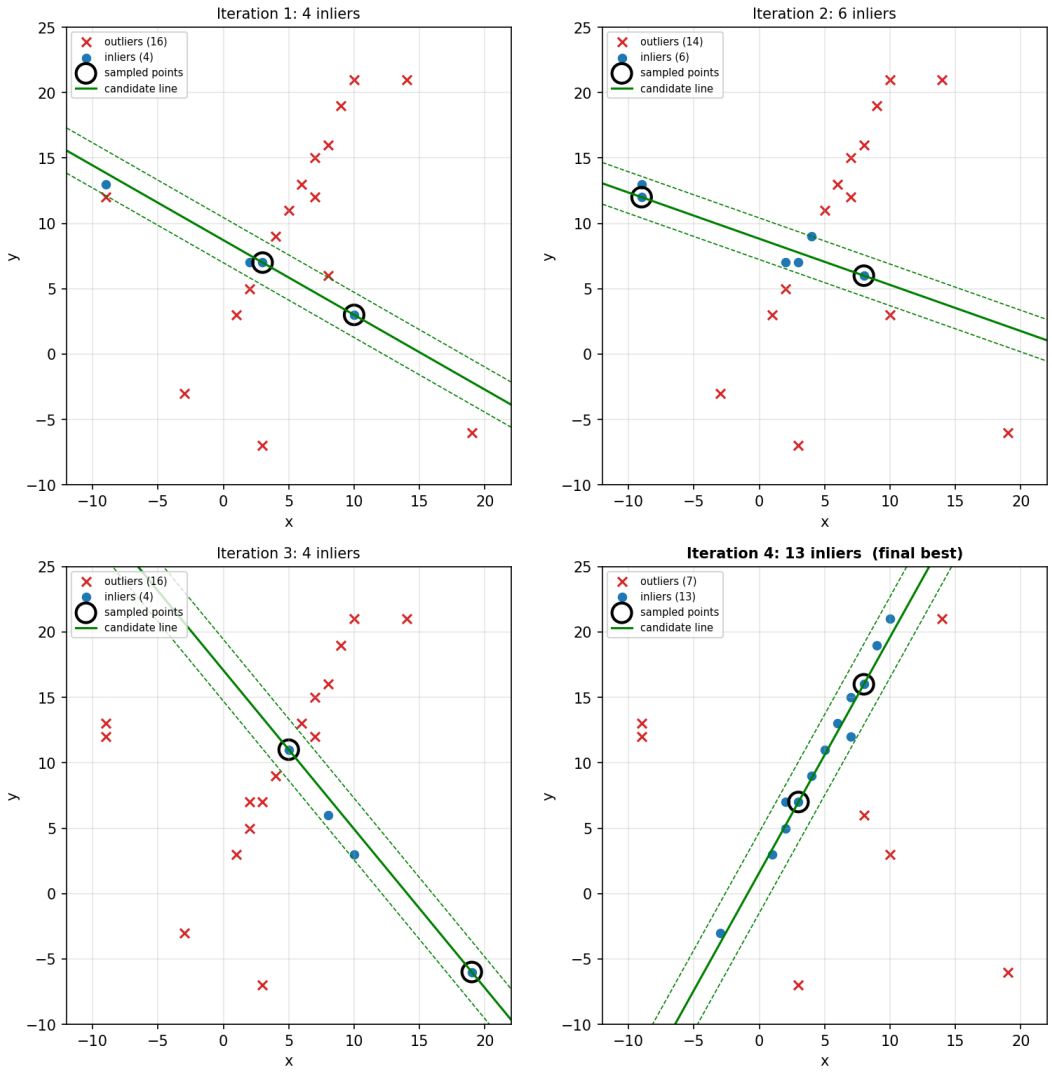


Figure 4: RANSAC iterations 1–4. Black circles mark the two sampled points, the green line is the candidate, and the dashed lines show the ± 1.5 inlier band. Iteration 4 is the final best model.

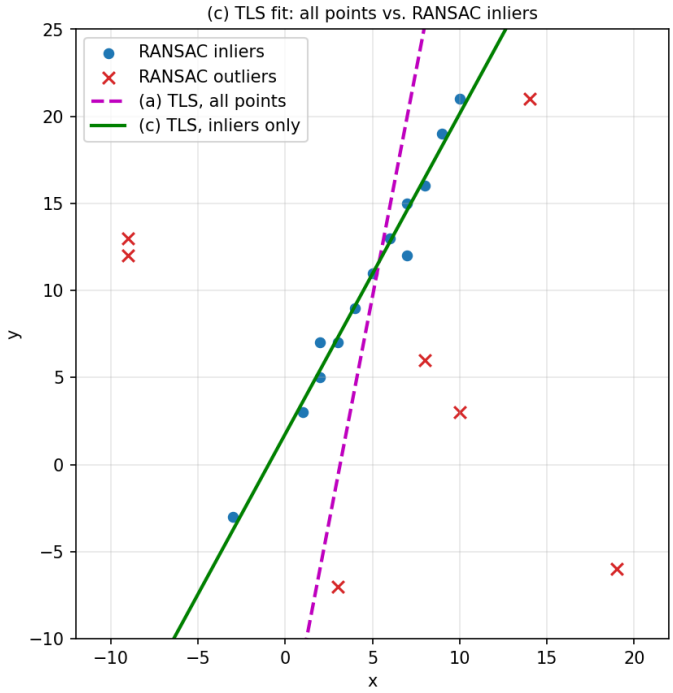
Final result:

- **13 inliers** and **7 outliers**. The outliers are $(19, -6)$, $(14, 21)$, $(8, 6)$, $(-9, 12)$, $(3, -7)$, $(-9, 13)$ and $(10, 3)$.
- Estimated inlier ratio: $w = 0.65$.
- **9 iterations** were performed in total.

(c)

TLS on the 13 inliers gives eigenvalues 0.214 and 53.47, and the line

$$-0.8785\,x + 0.4777\,y - 0.8386 = 0 \quad (y = 1.839x + 1.756).$$



The inlier-only line follows the main group of points closely: the RMS distance of the inliers is 0.46, compared with 2.30 for the line from (a). The all-points line from (a) is much steeper (slope 5.2 vs. 1.84) and misses most of the data.

The problem is that least squares squares every residual, so points far from the line have a very large influence. The 7 outliers (35% of the data) are spread far from the true line, for example $(19, -6)$, $(-9, 12)$ and $(3, -7)$. They inflate the covariance in every direction, so its two eigenvalues (42.7 and 63.6) are nearly equal, the "best" direction is poorly defined, and the fit is pulled away from the real structure. Once RANSAC removes the outliers, the smallest eigenvalue (0.21) is far below the largest (53.5), which shows a clear linear structure, and TLS recovers it accurately.

Appendix: Code

hw3_q4.py

```python
import math
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt

OUT = Path(__file__).resolve().parent / "figures"

P = np.array([
    (5, 11), (19, -6), (2, 5), (7, 15), (14, 21),
    (8, 6), (-9, 12), (9, 19), (6, 13), (3, -7),
    (4, 9), (10, 21), (3, 7), (-3, -3), (-9, 13),
    (8, 16), (10, 3), (2, 7), (7, 12), (1, 3),
], dtype=float)

THRESH = 1.5
CONF = 0.99
```

```
N_MAX = 100
S = 2
SEED = 0


def tls_fit(pts):
    mean = pts.mean(axis=0)
    cov = np.cov((pts - mean).T, bias=True)
    evals, evecs = np.linalg.eigh(cov)
    a, b = evecs[:, 0]
    if b < 0 or (b == 0 and a < 0):
        a, b = -a, -b
    c = -(a * mean[0] + b * mean[1])
    return np.array([a, b, c]), mean, cov, evals


def line_from_two(p1, p2):
    d = p2 - p1
    n = np.array([-d[1], d[0]]) / np.hypot(*d)
    return np.array([n[0], n[1], -n @ p1])


def distances(line, pts):
    return np.abs(pts @ line[:2] + line[2])


def required_iters(w):
    if w >= 1.0:
        return 1
    return math.ceil(math.log(1 - CONF) / math.log(1 - w ** S))


def ransac(pts, seed=SEED):
    rng = np.random.default_rng(seed)
    n = len(pts)
    best = None
    limit = N_MAX
    history = []
    it = 0
    while it < limit:
        it += 1
        i, j = rng.choice(n, size=2, replace=False)
        line = line_from_two(pts[i], pts[j])
        d = distances(line, pts)
        inl = d <= THRESH
        k = int(inl.sum())
        sse = float(np.sum(d[inl] ** 2))
        improved = best is None or k > best["k"]
        tie_better = best is not None and k == best["k"] and sse < best["sse"]
        rec = dict(it=it, idx=(i, j), line=line, inl=inl, k=k, sse=sse,
                   improved=improved, tie=tie_better, limit_before=limit)
        if improved or tie_better:
            best = rec
        if improved:
            w = k / n
            limit = min(required_iters(w), N_MAX)
        rec["limit_after"] = limit
        history.append(rec)
    return best, history


def line_xy(line, xlim):
    a, b, c = line
    if abs(b) > 1e-9:
        x = np.array(xlim)
        return x, -(a * x + c) / b
    return np.full(2, -c / a), np.array([-30, 30])

```

```python

XLIM = (-12, 22)
YLIM = (-10, 25)


def setup(ax, title):
    ax.set_xlim(*XLIM)
    ax.set_ylim(*YLIM)
    ax.set_aspect("equal")
    ax.grid(alpha=0.3)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_title(title, fontsize=10)


def plot_iteration(ax, rec, pts):
    inl = rec["inl"]
    ax.scatter(*pts[~inl].T, c="tab:red", marker="x", s=40, label=f"outliers ({(~inl).sum()})")
    ax.scatter(*pts[inl].T, c="tab:blue", s=30, label=f"inliers ({inl.sum()})")
    i, j = rec["idx"]
    ax.scatter(*pts[[i, j]].T, s=180, facecolors="none", edgecolors="k", linewidths=2,
               label="sampled points")
    ax.plot(*line_xy(rec["line"], XLIM), "g-", lw=1.5, label="candidate line")
    a, b, c = rec["line"]
    for s in (-1, 1):
        ax.plot(*line_xy((a, b, c + s * THRESH), XLIM), "g--", lw=0.8)
    setup(ax, f"Iteration {rec['it']}: {rec['k']} inliers")
    ax.legend(fontsize=7, loc="upper left")


def main():
    OUT.mkdir(exist_ok=True)

    line_a, mean, cov, evals = tls_fit(P)
    print("(a) TLS on all 20 points")
    print(f" mean = {mean}, covariance =\n{cov}\n eigenvalues = {evals}")
    print(f" line: {line_a[0]:.4f} x + {line_a[1]:.4f} y + {line_a[2]:.4f} = 0, "
          f"slope = {-line_a[0] / line_a[1]:.4f}, intercept = {-line_a[2] / line_a[1]:.4f}")

    fig, ax = plt.subplots(figsize=(6, 6))
    ax.scatter(*P.T, c="k", s=25, label="data")
    ax.plot(*line_xy(line_a, XLIM), "m-", lw=2, label="TLS (all points)")
    setup(ax, "(a) Orthogonal least squares, all 20 points")
    ax.legend()
    fig.savefig(OUT / "q4a_tls_all.png", dpi=150, bbox_inches="tight")
    plt.close(fig)

    best, hist = ransac(P)
    print("\n(b) RANSAC log")
    for r in hist:
        flag = " <- new best" if r["improved"] else (" <- tie, smaller SSE" if r["tie"] else "")
        print(f" it {r['it']:2d}: pts {r['idx']}, inliers {r['k']:2d}, SSE {r['sse']:.3f}, "
              f"limit {r['limit_after']}{flag}")
    n_in = best["k"]
    print(f" final: inliers = {n_in}, outliers = {len(P) - n_in}, w = {n_in / len(P):.2f}, "
          f"iterations = {len(hist)}, best found at iteration {best['it']}")
    print(f" outlier points: {P[~best['inl']].tolist()}")

    shown = [hist[0]]
    for r in hist:
        if r["improved"] and r["it"] not in [s["it"] for s in shown]:
            shown.append(r)
    rest = [r for r in hist if r["it"] not in [s["it"] for s in shown]]
    if rest:
        shown.append(max(rest, key=lambda r: (not r["tie"] and r["k"] < best["k"], -r["k"])))
    if best["it"] not in [s["it"] for s in shown]:
        shown.append(best)
    shown = sorted(shown, key=lambda r: r["it"])
```

```
    cols = 2 if len(shown) == 4 else min(len(shown), 3)
    rows = math.ceil(len(shown) / cols)
    fig, axes = plt.subplots(rows, cols, figsize=(5.2 * cols, 5.2 * rows), squeeze=False)
    for ax, r in zip(axes.flat, shown):
        plot_iteration(ax, r, P)
        if r["it"] == best["it"]:
            ax.set_title(ax.get_title() + " (final best)", fontsize=10, fontweight="bold")
    for ax in axes.flat[len(shown):]:
        ax.axis("off")
    fig.tight_layout()
    fig.savefig(OUT / "q4b_ransac_iterations.png", dpi=150, bbox_inches="tight")
    plt.close(fig)
    print(f" plotted iterations: {[r['it'] for r in shown]}")

    line_c, mean_c, cov_c, evals_c = tls_fit(P[best["inl"]])
    print("\n(c) TLS on RANSAC inliers")
    print(f" mean = {mean_c}, eigenvalues = {evals_c}")
    print(f" line: {line_c[0]:.4f} x + {line_c[1]:.4f} y + {line_c[2]:.4f} = 0, "
          f"slope = {-line_c[0] / line_c[1]:.4f}, intercept = {-line_c[2] / line_c[1]:.4f}")
    for name, ln in [("all-points TLS", line_a), ("inlier TLS", line_c)]:
        d_in = distances(ln, P[best["inl"]])
        print(f" {name}: RMS perpendicular distance of inliers = {np.sqrt(np.mean(d_in ** 2)):.3f}")

    fig, ax = plt.subplots(figsize=(6.5, 6.5))
    ax.scatter(*P[best["inl"]].T, c="tab:blue", s=30, label="RANSAC inliers")
    ax.scatter(*P[~best["inl"]].T, c="tab:red", marker="x", s=50, label="RANSAC outliers")
    ax.plot(*line_xy(line_a, XLIM), "m--", lw=2, label="(a) TLS, all points")
    ax.plot(*line_xy(line_c, XLIM), "g-", lw=2, label="(c) TLS, inliers only")
    setup(ax, "(c) TLS fit: all points vs. RANSAC inliers")
    ax.legend(loc="upper left")
    fig.savefig(OUT / "q4c_comparison.png", dpi=150, bbox_inches="tight")
    plt.close(fig)


if __name__ == "__main__":
    main()
```